

CS 3502 OS Project Report

Multi-Threaded Banking System

Leandro Cherulli

07/13/2026

1 Introduction

This project demonstrates important multi-threading concepts using a banking system written in C with threads. The project is divided into four phases: basic threading, mutex synchronization, deadlock creation, and deadlock prevention.

Multiple teller threads access shared bank account information. Each phase shows a different problem or solution related to concurrent programming.

2 Phase 1: Race Condition

In Phase 1, four teller threads perform deposits into the same shared account. No mutex locks are used. Each thread reads the current balance, modifies it, and writes the new value back.

Because multiple threads may read the same balance before another thread updates it, some deposits are overwritten. This produces an incorrect final balance and transaction count.

2.1 Result

The expected final balance is calculated using:

$$ExpectedBalance = InitialBalance + (Threads \times Operations \times DepositAmount)$$

The actual balance is usually lower because some updates are lost.

```
leandro@leandro-VMware-Virtual-Platform:~/cs3502/multi-thread-banking-project$ ./phase1
Phase 1: Threads Without Synchronization
Initial balance: 1000.00
Number of deposits: 4000

Teller 1, thread 128652568164032: deposit #1, balance 1000.00 -> 1001.00
Teller 1, thread 128652568164032: deposit #2, balance 1001.00 -> 1002.00
Teller 2, thread 128652559771328: deposit #1, balance 1001.00 -> 1002.00
Teller 4, thread 128652542985920: deposit #1, balance 1001.00 -> 1002.00
Teller 3, thread 128652551378624: deposit #1, balance 1001.00 -> 1002.00
Teller 4, thread 128652542985920: deposit #2, balance 1002.00 -> 1003.00
Teller 2, thread 128652559771328: deposit #2, balance 1002.00 -> 1003.00
Teller 3, thread 128652551378624: deposit #2, balance 1003.00 -> 1004.00
Teller 4, thread 128652542985920: deposit #1000, balance 2000.00 -> 2001.00
Teller 3, thread 128652551378624: deposit #1000, balance 2001.00 -> 2002.00
Teller 2, thread 128652559771328: deposit #1000, balance 2000.00 -> 2001.00
Teller 1, thread 128652568164032: deposit #1000, balance 1999.00 -> 2000.00

Expected final balance: 5000.00
Actual final balance: 2000.00
Expected transaction count: 4000
Actual transaction count: 1000

Race condition demonstrated: some updates were lost.
```

Figure 1: Phase 1 output showing a race condition

3 Phase 2: Mutex Protection

Phase 2 adds a mutex to the shared account. Before changing the account balance or transaction count, a teller thread must lock the mutex.

Only one thread can enter the critical section at a time. After completing the deposit, the thread unlocks the mutex so another thread can access the account.

The important synchronization operations are:

```
1 pthread_mutex_lock(&account.lock);
2
3 account.balance += amount;
4 account.transaction_count++;
5
6 pthread_mutex_unlock(&account.lock);
```

3.1 Result

The final balance and transaction count match the expected values because the mutex prevents lost updates.

```

leandro@leandro-VMware-Virtual-Platform:~/cs3502/multi-thread-banking-project$ ./phase2
Phase 2: Mutex-Protected Account
Initial balance: 1000.00

Teller 1, thread 139817734108864: deposit #1 completed; balance = 1001.00
Teller 1, thread 139817734108864: deposit #2 completed; balance = 1002.00
Teller 1, thread 139817734108864: deposit #1000 completed; balance = 2000.00
Teller 3, thread 139817717323456: deposit #1 completed; balance = 2001.00
Teller 2, thread 139817725716160: deposit #1 completed; balance = 2002.00
Teller 2, thread 139817725716160: deposit #2 completed; balance = 2003.00
Teller 2, thread 139817725716160: deposit #1000 completed; balance = 3001.00
Teller 3, thread 139817717323456: deposit #2 completed; balance = 3002.00
Teller 4, thread 139817708930752: deposit #1 completed; balance = 3778.00
Teller 4, thread 139817708930752: deposit #2 completed; balance = 3779.00
Teller 4, thread 139817708930752: deposit #1000 completed; balance = 4884.00
Teller 3, thread 139817717323456: deposit #1000 completed; balance = 5000.00

Expected final balance: 5000.00
Actual final balance: 5000.00
Expected transaction count: 4000
Actual transaction count: 4000
Execution time: 0.000588 seconds

Synchronization successful: no updates were lost.

```

Figure 2: Phase 2 output showing successful synchronization

4 Phase 3: Deadlock Creation

Phase 3 uses two accounts and two transfer threads.

Thread 1 locks Account 0 and then waits for Account 1. At the same time, Thread 2 locks Account 1 and waits for Account 0.

This creates circular wait:

- Thread 1 holds Account 0 and waits for Account 1.
- Thread 2 holds Account 1 and waits for Account 0.

Neither thread can continue because each thread is waiting for a mutex held by the other thread.

A watchdog thread waits for three seconds and then reports that the program is dead-locked.

```

leandro@leandro-VMware-Virtual-Platform:~/cs3502/multi-thread-banking-project$ ./phase3
Phase 3: Intentional Deadlock Demonstration
Account 0 balance: 1000.00
Account 1 balance: 1000.00

Thread 1 (136338149996224): attempting transfer 100.00 from account 0 to account 1
Thread 1: locked account 0
Thread 2 (136338141603520): attempting transfer 100.00 from account 1 to account 0
Thread 2: locked account 1
Thread 2: waiting to lock account 0...
Thread 1: waiting to lock account 1...

WATCHDOG: No transfer completed for 3 seconds.
WATCHDOG: Possible deadlock detected.
Thread 1 holds account 0 and waits for account 1.
Thread 2 holds account 1 and waits for account 0.
The program will now terminate the intentional deadlock demonstration.

```

Figure 3: Phase 3 output showing the intentional deadlock

5 Phase 4: Deadlock Prevention

Phase 4 prevents deadlock by using consistent lock ordering.

Before performing a transfer, the program determines which account has the lower account ID. Every thread locks the lower account ID first and the higher account ID second.

```

1 int first = (from_id < to_id) ? from_id : to_id;
2 int second = (from_id < to_id) ? to_id : from_id;
3
4 pthread_mutex_lock(&accounts[first].lock);
5 pthread_mutex_lock(&accounts[second].lock);

```

Because every thread follows the same locking order, circular wait cannot occur. After the transfer is complete, the mutexes are unlocked in reverse order.

5.1 Result

All transfer threads finish successfully. The combined balance of both accounts remains unchanged, showing that no money was created or lost.

```

leandro@leandro-VMware-Virtual-Platform:~/cs3502/multi-thread-banking-project$ ./phase4
Phase 4: Deadlock Prevention With Lock Ordering
Initial account 0 balance: 10000.00
Initial account 1 balance: 10000.00
Initial combined balance: 20000.00

Thread 1 (137664745436864): transfers account 0 -> account 1
Thread 2 (137664737044160): transfers account 1 -> account 0
Thread 3 (137664728651456): transfers account 0 -> account 1
Thread 4 (137664720258752): transfers account 1 -> account 0
Thread 5 (137664711866048): transfers account 0 -> account 1
Thread 6 (137664703473344): transfers account 1 -> account 0
Thread 1 completed 10000 successful transfers.
Thread 3 completed 10000 successful transfers.
Thread 2 completed 10000 successful transfers.
Thread 5 completed 10000 successful transfers.
Thread 6 completed 10000 successful transfers.
Thread 4 completed 10000 successful transfers.

Final account 0 balance: 10000.00
Final account 1 balance: 10000.00
Final combined balance: 20000.00
Successful transfers: 60000
Account 0 transaction count: 60000
Account 1 transaction count: 60000
Execution time: 0.007535 seconds

Success: all threads finished without deadlock.
The combined account balance remained unchanged.

```

Figure 4: Phase 4 output showing deadlock prevention

6 Challenges and Solutions

One challenge was making the race condition visible in Phase 1. Small delays and thread scheduling calls were added between reading and writing the shared values. This increased the chance that multiple threads would access the same data simultaneously.

Another challenge was creating a reliable deadlock in Phase 3. A thread barrier was used so that both threads locked their first account before attempting to lock the second account.

The Phase 4 deadlock was resolved by applying a consistent lock-ordering rule.

7 Performance Observations

Phase 1 does not use mutex locks, but its results are unreliable because transactions can be lost.

Phase 2 adds synchronization overhead because threads must wait for access to the shared account. However, the final results are correct.

Phase 3 does not complete normally because the threads become permanently blocked.

Phase 4 completes successfully. The ordered-locking strategy adds some waiting time, but it prevents deadlock and maintains correct account balances.

8 Conclusion

This project demonstrated the importance of synchronization in multi-threaded programs. Without synchronization, race conditions can produce incorrect results. Mutex locks protect critical sections but can also create deadlock when multiple resources are acquired in an inconsistent order.

Using consistent lock ordering prevents circular wait and allows concurrent transactions to complete safely.